多态综合案例

潘茂林

panml@mail.sysu.edu.cn

13902210963

计算机学院, 广州

*

Polymorphism（案例）

简单菜单制作

结构化方法的菜单程序

- 请使用关键词“**c++ 制作 menu**”搜索

- 鉴于结果太多，程序大致伪代码是：

```
Display menu title
Display menu items
Input your choice
Switch (choice) {
    choice A: doSomething();break;
    …
}
```

结构化程序就是将一组动作（函数）按顺序、分支或循环结构进行组织。

注意：

1. 所有语句动词开头
2. 如果动作复杂，再展开成更细粒度的动作

面向对象的程序是？

面向对象的菜单程序

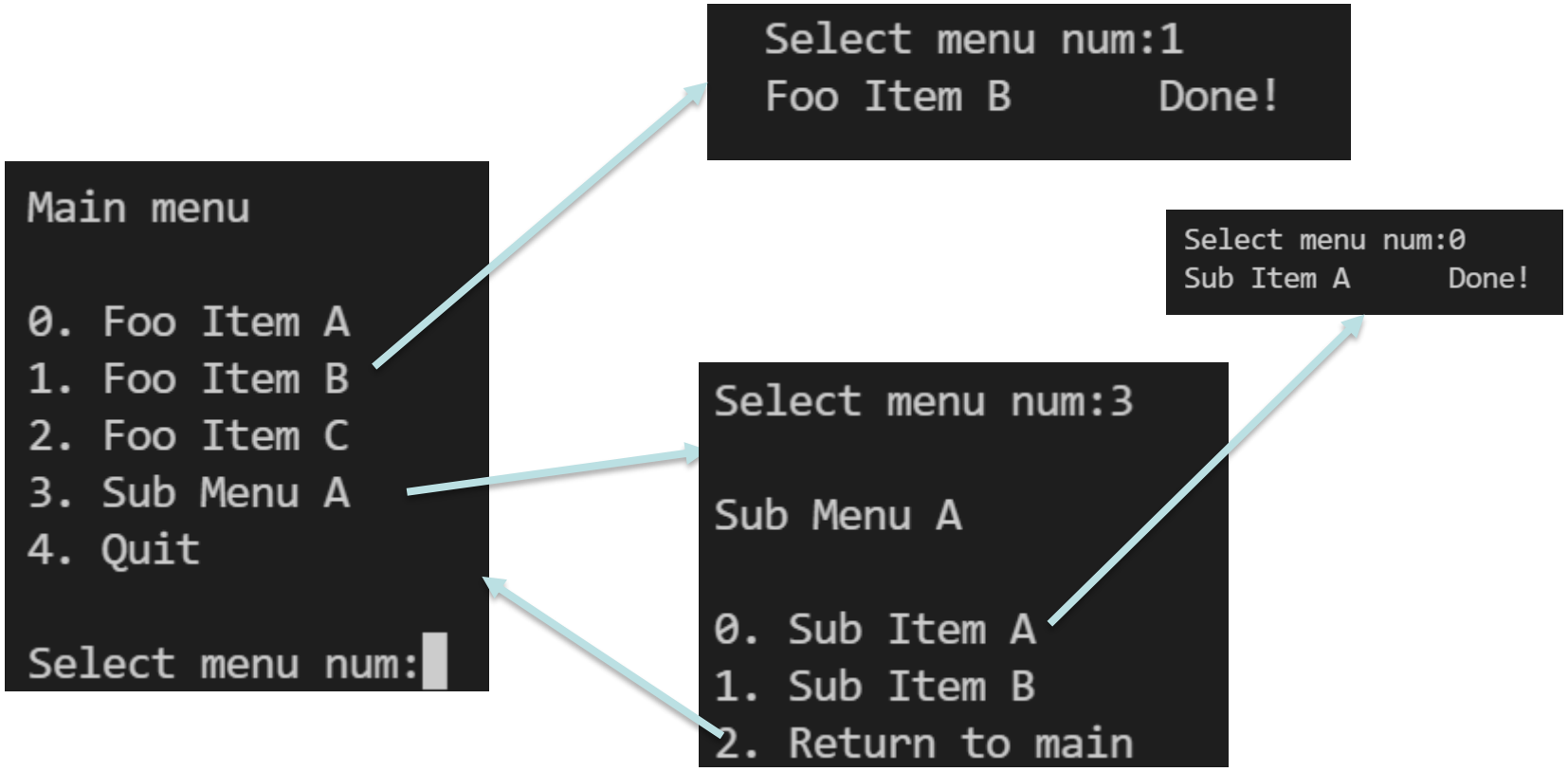
```
#define NEW(x) *(new x)
#define DEL(x) delete &x

int main() {
    Menu &root = NEW(Menu("Main menu"));
    root.addItem(NEW(FooMenuItem("Foo Item A")));
    root.addItem(NEW(FooMenuItem("Foo Item B")));
    root.addItem(NEW(FooMenuItem("Foo Item C")));
    AbstractMenu &sub = root.addItem(NEW(Menu("Sub Menu A")));
        Menu &subMenu = dynamic_cast<Menu &>(sub);
        subMenu.addItem(NEW(FooMenuItem("Sub Item A")));
        subMenu.addItem(NEW(FooMenuItem("Sub Item B")));
        subMenu.addItem(NEW(QuitMenuItem("Return to main")));
    root.addItem(NEW(QuitMenuItem("Quit")));

    root.execute();
    DEL(root);
    return 0;
}
```

面向对象的程序有些像做玩具汽车。首先要一堆零件，然后按说明书拼装起来，最后按发动按钮。
你也可以将自己想成将军。战役之前，你先得按作战图排兵布阵，再启动战役。
所有语句是以物体（对象）为中心

前面程序的执行结果


Main menu
0. Foo Item A
1. Foo Item B
2. Foo Item C
3. Sub Menu A
4. Quit
Select menu num:
Select menu num:1
Foo Item B Done!
Select menu num:3
Sub Menu A
0. Sub Item A
1. Sub Item B
2. Return to main
Select menu num:0
Sub Item A Done!

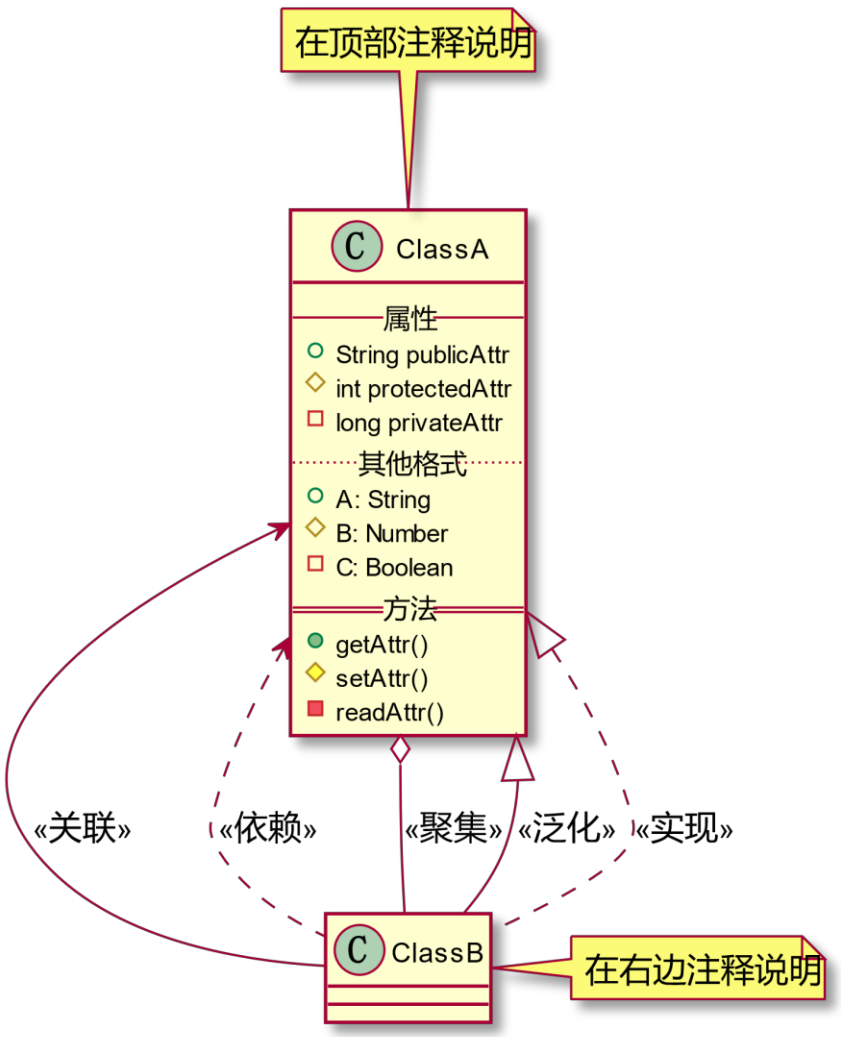
统一建模语言（起步）

（Unified Modeling Language，UML）

类与类之间关系的表示

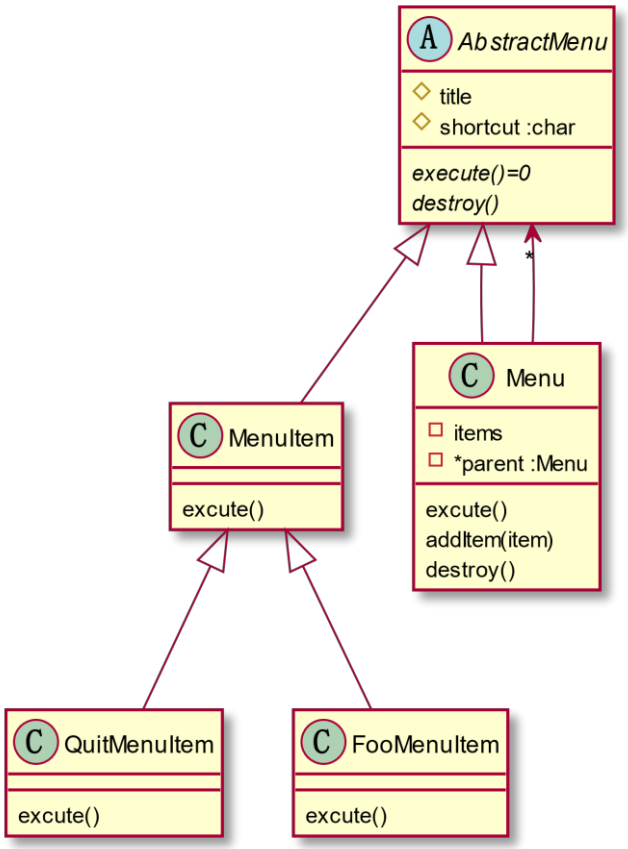
menu > design.puml > ...

```
1  @startuml test
2
3  Class ClassA {
4      -- 属性 --
5      +String publicAttr
6      #int protectedAttr
7      -long privateAttr
8      .. 其他格式 ..
9      +A: String
10     #B: Number
11     -C: Boolean
12     == 方法 ==
13     +getAttr()
14     #setAttr()
15     -readAttr()
16 }
17
18 note top: 在顶部注释说明
19
20 Class ClassB {
21
22 }
23
24 note right: 在右边注释说明
25
26 ClassA <-- ClassB:<<关联>>
27 ClassA <.. ClassB:<<依赖>>
28 ClassA o-- ClassB:<<聚集>>
29 ClassA <|-- ClassB:<<泛化>>
30 ClassA <|.. ClassB:<<实现>>
31 @enduml
```



Powered by PlantUML https://plantuml.com/zh/

菜单类的设计



menu > design.puml > {} Menu

```
1  @startuml Menu
2  abstract class  AbstractMenu{
3      # title
4      # shortcut :char
5      {abstract} execute()=0
6      {abstract} destroy()
7  }
8
9  class MenuItem {
10     {method} excute()
11 }
12
13 class QuitMenuItem {
14     {method} excute()
15 }
16
17 class FooMenuItem {
18     {method} excute()
19 }
20
21 class Menu {
22     - items
23     - *parent :Menu
24     {method} excute()
25     addItem(item)
26     destroy()
27 }
28
29 AbstractMenu ^-- MenuItem
30 MenuItem ^-- QuitMenuItem
31 MenuItem ^-- FooMenuItem
32 AbstractMenu ^-- Menu
33 Menu --> "*" AbstractMenu
34 @enduml
```

读图

- 抽象菜单
  - 菜单都有标题，快捷键属性
  - 菜单都能被执行，但怎么执行不知道
  - 抽象菜单指针可能析构具体菜单
- 菜单项
  - 菜单项知道具体执行任务
  - Quit菜单项用于返回上一级菜单
  - Foo菜单项是给用户的菜单编程Demo
- 菜单
  - 菜单包含一组菜单项或子菜单，并知道父菜单
  - 菜单执行的任务是显示菜单，并执行用户选择的菜单项
  - 提供菜单项管理服务，如 addItem，deleteItem，getItem等

编程知识与技巧

Main()代码

```
#define NEW(x) *(new x)
#define DEL(x) delete &x

int main() {
    Menu &root = NEW(Menu("Main menu"));
    root.addItem(NEW(FooMenuItem("Foo Item A")));
    root.addItem(NEW(FooMenuItem("Foo Item B")));
    root.addItem(NEW(FooMenuItem("Foo Item C")));
    AbstractMenu &sub = root.addItem(NEW(Menu("Sub Menu A")));
        Menu &subMenu = dynamic_cast<Menu &>(sub);
        subMenu.addItem(NEW(FooMenuItem("Sub Item A")));
        subMenu.addItem(NEW(FooMenuItem("Sub Item B")));
        subMenu.addItem(NEW(QuitMenuItem("Return to main")));
    root.addItem(NEW(QuitMenuItem("Quit")));

    root.execute();
    DEL(root);
    return 0;
}
```

使用指针，引用？

- 你必须形成自己的风格
  - 建议访问对象使用引用
    - 除非这个变量可能为空
    - 引用不能空值，会引发异常
  - 如果你的团队都习惯使用指针
    - 除非API库提供的函数返回是引用
  - 特别注意：

```
AbstractMenu &sub = root.addItem(NEW(Menu("Sub Menu A")));
    Menu &subMenu = dynamic_cast<Menu &>(sub);
```

Dynamic Cast

dynamic_cast <type-id> (expression)

- 该运算符把**expression**地址转换成**type-id**类型的地址。
- **Type-id** 必须是类的指针、类的引用或者 **void***；
- **dynamic_cast**在多态时，利用**RTTI**机制实现父类指针向下安全转换为子类型指针
  - 如果是指针向下类型转换失败，返回空值
  - 如果是引用向下类型转换失败，引发异常

Dynamic Cast

```
AbstractMenu& addItem(AbstractMenu &item) {
    items[itemCounter++] = &item;
    try {
        Menu &menu = dynamic_cast<Menu &>(item);
        menu.parent = this;
    } catch (bad_cast) {};
    return item;
};
```

item是父类型引用
menu是子类型的引用
必须用 try 处理异常

注意：在main中的dynamic转换不可能引发异常，因此不用 try

抽象菜单

```
class AbstractMenu {
    public:
        virtual void execute() = 0;
        virtual ~ AbstractMenu() {};
        AbstractMenu(const string title, char shortcut = 0):
            title(title),shortcut(shortcut) {};
        string& getTitle() {return title;};
    protected:
        string title;
        char shortcut;
};
```

要点：

1. 纯虚函数 execute。说明这是抽象类，不可能有这样的实例
2. 虚析构。使用该指针可析构它子类的任何实例对象
3. 你可能需要补充其他方法，如 getShortcut 等 get/set 函数成员

普通菜单项

```
class MenuItem: public AbstractMenu {
    public:
        void execute() {};
        MenuItem(const string title, char shortcut = 0):
            AbstractMenu(title,shortcut) {};
};

class QuitMenuItem:public MenuItem {
    public:
        QuitMenuItem(const string title, char shortcut = 'q'):
            MenuItem(title,shortcut) {};
};

class FooMenuItem:public MenuItem {
    public:
        void execute() {cout << title << "\tDone!" << endl;};
        FooMenuItem(const string title, char shortcut = 0):
            MenuItem(title,shortcut) {};
};
```

要点：1. QuitMenuItem；2. 用户定义自己的菜单项，需要重写 execute

菜单、子菜单（1）

```
Menu(const string title, char shortcut = 0, int maxitems = 10):
    AbstractMenu(title,shortcut) {
        items = new AbstractMenu *[maxitems];
        itemCounter = 0;
        parent = NULL;
    };
~ Menu() {
    //Todo: BUG
    delete items;
    cout << "Items left in memory" << endl;
};
AbstractMenu& addItem(AbstractMenu &item) {
    items[itemCounter++] = &item;
    try {
        Menu &menu = dynamic_cast<Menu &>(item);
        menu.parent = this;
    } catch (bad_cast) {};
    return item;
};
```

要点：

1. 构造分配空间 **items
2. parent指针一定要初始化
3. 析构要完善，自动释放所有菜单项对象
4. 向下转换引用类型，必须处理异常

菜单、子菜单（2）

```
void execute() {
    while (true) {
        _displayTitle();
        _displayItemList();
        int selected;
        cout << endl << "Select menu num:";
        cin >> selected;
        //Todo: BUG
        AbstractMenu *selectedItem = items[selected];
        if (typeid(QuitMenuItem)==typeid(*selectedItem)) break;
        selectedItem->execute();
    }
};
```

要点：

1. 使用私有方法，使得程序更加可读
2. 可能使用 char 接受用户输入更佳
3. 必须检查输入，防止访问非法的item。这里没有
4. 使用typeid关键字处理特殊的菜单项，如QuitMenuItem
5. 普通菜单项和菜单执行execute。

为什么要这样设计？

- 你已经应用了一种设计模式
  - 组合模式，用于构建树状数据结构
- 组合模式应用案例
  - 文件系统。抽象文件，普通文件，目录，特殊文件（如**Link**，**Device**等）
  - 装配清单：抽象装配件，零件，部件

练习

- 请你完善菜单编程框架
  1. **new FooMenuItem(“Open &File”);** 创建的菜单项，支持**f**，**F**作为快捷键
  2. **delete *root**；应释放自己及其子菜单的所有**Item**
  3. **new GroupMenuItem(“-”)**；创建分割线这样的特殊菜单项
  4. 其他你认为需要改进的地方？

思考

- 对比结构化方法与面向对象方法编写的代码，请分别给出优点和缺点